A Lightweight and Tight-Binding Post-Quantum Authenticated Key Exchange for IoT using ML-KEM

Khiem Pham Tuan[1,2†], Dong Bui Huu[3,4†], Tot Tran Dac[2,5*]

[1]PhD student in Computer Science, Ho Chi Minh City Open University, 97 Vo Van Tan Street, Xuan Hoa Ward, Ho Chi Minh City, 700000, Viet Nam.
[2]Lecturer, Faculty of Information Technology, Ho Chi Minh City University of Industry and Trade (HUIT), 140 Le Trong Tan Street, Tay Thanh Ward, Ho Chi Minh City, 700000, Viet Nam.
[3]PhD student in Computer Science, Ho Chi Minh City University of Technology (HCMUT), 268 Ly Thuong Kiet Street, Dien Hong Ward, Ho Chi Minh City, 700000, Viet Nam.
[4]Lecturer, Faculty of Management Information Systems, Ho Chi Minh University of Banking, 36 Ton That Dam Street, Sai Gon Ward, Ho Chi Minh City, 700000, Viet Nam.
[5]PhD student in Information Security, Academy of Cryptography Techniques, 141 Chien Thang Street, Thanh Liet Ward, Ha Noi City, 700000, Viet Nam.

*Corresponding author(s). E-mail(s): ncsat0502@actvn.edu.vn; tottd@huit.edu.vn;
Contributing authors: khiempt.25ai@ou.edu.vn; khiempt@huit.edu.vn; dong.buihuu@hcmut.edu.vn; dongbh@hub.edu.vn;
†These authors contributed equally to this work.

Abstract

The rapid transition towards Post-Quantum Cryptography (PQC) poses significant integration challenges for resource-constrained Internet of Things (IoT) devices. While the standardized Module-Lattice-Based Key-Encapsulation Mechanism (ML-KEM) offers robust protection against quantum threats, combining it with post-quantum digital signatures (e.g., ML-DSA) for authentication

introduces prohibitive computational delays, high memory consumption, and severe vulnerability to Denial-of-Service (DoS) attacks. To address these critical bottlenecks, this paper proposes TB-PQ-AKE (Token-Based Post-Quantum Authenticated Key Exchange), a highly optimized, purely KEM-based protocol tailored for IoT edge devices. By replacing heavy digital signatures with implicit mutual authentication using a symmetric Master Secret Key and lightweight cryptographic hashing (KMAC/HKDF), the proposed protocol significantly reduces overhead. Furthermore, TB-PQ-AKE integrates an "authenticate first, compute later" token mechanism that drops malicious requests at the earliest stage, ensuring strong DoS resilience. Security analysis demonstrates that the protocol achieves implicit mutual authentication, forward secrecy, and replay attack resistance under the standard Dolev-Yao threat model. Hardware-level profiling on the ARM Cortex-M4 microcontroller validates its extreme efficiency, requiring only 785,000 CPU cycles, 83.7 KB of Flash, and an ultra-low 2.17 KB of RAM. Comparative results confirm that TB-PQ-AKE outperforms traditional PQ-PKI, Post-Quantum TLS 1.3 PSK, and state-of-the-art hybrid protocols, proving its practicality for next-generation quantum-safe IoT networks.

Keywords: Post-Quantum Cryptography, ML-KEM, Authenticated Key Exchange, IoT Security, Man-In-The-Middle, Implicit Authentication.

1 Introduction

1.1 Background

The rapid proliferation of Internet of Things (IoT) devices has led to a massive increase in sensitive data transmission over insecure public channels. To establish secure communication, an Authenticated Key Exchange (AKE) protocol is essential to provide mutual authentication and derive a symmetric shared session key. Currently, standard protocols such as Transport Layer Security (TLS) version 1.3 [1] are widely deployed to secure these channels, relying heavily on traditional public-key cryptography algorithms like RSA and Elliptic Curve Cryptography (ECC).

1.2 The Quantum Threat

However, the foundational security of these traditional public-key cryptosystems is severely threatened by the advent of quantum computing. In 1994, Peter Shor proposed a groundbreaking quantum algorithm capable of solving integer factorization and discrete logarithm problems in polynomial time [2]. Consequently, a sufficiently powerful, large-scale quantum computer could easily break widely used algorithms like RSA and ECC. This renders current IoT communications highly vulnerable, particularly to "Store Now, Decrypt Later" (SNDL) attacks, creating an urgent mandate to transition to Post-Quantum Cryptography (PQC).

1.3 The Problem with PQC in IoT

In response to the quantum threat, the National Institute of Standards and Technology (NIST) has aggressively driven the standardization of PQC, selecting lattice-based algorithms like CRYSTALS-Kyber (now officially ML-KEM) for key encapsulation [3]. While ML-KEM provides robust quantum resistance, deploying PQC in resource-constrained IoT environments introduces significant hurdles. Edge IoT devices typically operate with severe limitations in processing power, memory, and energy. PQC algorithms inherently demand larger key sizes and higher computational overhead compared to their classical counterparts.

This performance bottleneck is especially pronounced when utilizing PQC digital signatures for authentication, which are exceptionally heavy and introduce substantial communication latency. To mitigate this, recent research trends, such as the KEMTLS protocol [4], advocate for KEM-based authentication—replacing expensive post-quantum signatures entirely with Key Encapsulation Mechanisms (KEMs) during the handshake. While this improves efficiency, deploying such protocols directly in IoT environments without robust Denial-of-Service (DoS) protection remains dangerous, as malicious actors can easily exhaust the limited resources of IoT devices by initiating excessive, unverified cryptographic requests.

1.4 Contributions

To address the critical intersection of post-quantum security and IoT resource constraints, this paper proposes TB-PQ-AKE (Token-Based Post-Quantum Authenticated Key Exchange), a highly optimized, quantum-safe protocol. The main contributions of our work are summarized as follows:

- We propose a lightweight, purely KEM-based AKE protocol that completely eliminates the need for resource-intensive post-quantum digital signatures, significantly reducing computational delays and communication overhead for constrained devices.
- We design and integrate a lightweight Token-based pre-authentication mechanism. This allows devices to perform early verification ("authenticate first, compute later"), effectively dropping malicious or forged requests to prevent resource-exhaustion (DoS) attacks before heavy KEM operations are executed.
- We comprehensively analyze the security properties of the proposed protocol, demonstrating that TB-PQ-AKE ensures strong implicit mutual authentication, forward secrecy, and resilience against active attacks (e.g., Replay and Man-in-the-Middle) under the standard Dolev-Yao threat model.

2 Related Works

2.1 The Transition to PQC and Challenges for IoT

The rapid development of quantum computing technology poses a severe threat to the security foundation of traditional public-key cryptographic systems such as RSA and ECC. To counter this prospect, the National Institute of Standards and Technology (NIST) has officially standardized Post-Quantum Cryptography (PQC) algorithms,

notably ML-KEM for key encapsulation mechanisms and ML-DSA for digital signatures [5]. However, the direct application of these PQC algorithms to the Internet of Things (IoT) environment encounters significant obstacles. Edge IoT devices are typically strictly constrained in processing power, memory capacity, and energy, whereas PQC algorithms demand large key sizes and heavy computational workloads [6]. Extensive comparative studies, such as the benchmarking of various PQC primitives on IoT nodes by Septien-Hernandez et al. [7], explicitly highlight these severe memory and processing bottlenecks. This creates an urgent need to optimize cryptographic protocols specifically for quantum-safe IoT networks.

2.2 Hybrid Key Exchange Protocols (Hybrid PQC AKE)

To balance long-term security and practical deployability, the hybrid key exchange (Hybrid AKE) model is attracting significant attention from the research community. A common approach is combining a post-quantum KEM mechanism with digital signatures (either traditional or post-quantum) for authentication. Recently, Battarbee et al. [8] proposed the Muckle+ protocol, a HAKE solution designed for large-scale networks. To achieve authentication, Muckle+ utilizes post-quantum digital signatures. However, as the authors themselves pointed out, the performance of current post-quantum signature schemes (such as ML-DSA) still lags significantly behind their ML-KEM counterparts [8]. The use of heavy digital signatures increases transaction latency and consumes substantial energy, making solutions like Muckle+ sub-optimal for ultra-lightweight IoT devices. This limitation is also evident in highly recent domain-specific adaptations. For instance, the AmphiKey protocol [9] designed for Smart Grids integrates a dual-mode hybrid AKE but still fundamentally relies on the Raccoon Digital Signature Algorithm for strong authentication, thereby inheriting the associated post-quantum signature overhead.

2.3 Lightweight Authentication and DoS Resistance

To overcome the performance barriers of PQC digital signatures, current design trends lean towards completely eliminating digital signatures during the key exchange process. Instead, authentication can be implicitly guaranteed (implicit authentication) through the combination of KEM algorithms and high-performance symmetric primitives like Keyed Message Authentication Codes (MAC) [10]. Furthermore, protecting IoT devices from resource exhaustion attacks (DoS/DDoS) right from the initial handshake step is a critical factor. The EDHOC standard (RFC 9528) [11] establishes an excellent design philosophy for IoT networks: utilizing concise tokens and implementing an "authenticate first, compute later" approach. Recent research has attempted to quantum-safe this standard, such as the PQ-EDHOC evaluation by Fraile et al. [12]. However, directly integrating PQC into existing generalized standards often retains complex message flows. Inheriting these concepts, our proposed TB-PQ-AKE protocol aims to bridge the research gap: a purely KEM-based AKE protocol (KEM-based authentication) that omits digital signatures to optimize performance for IoT devices, while simultaneously integrating a Token mechanism to thwart DoS attacks early on.

3 Preliminaries

This section establishes the mathematical foundations and cryptographic standards utilized in the proposed TB-PQ-AKE protocol.

3.1 Module-Lattice-Based Key-Encapsulation Mechanism (ML-KEM)

ML-KEM is a lattice-based key encapsulation mechanism standardized in NIST FIPS 203 [13], which relies on the computational hardness of the Module Learning with Errors (MLWE) problem. An ML-KEM scheme is defined by a triplet of algorithms:

- ML-KEM.KeyGen() $\rightarrow (ek, dk)$: A randomized algorithm that generates a public encapsulation key ek and a private decapsulation key dk.
- ML-KEM.Encaps(ek) $\rightarrow (ct, ss)$: Uses the encapsulation key ek to generate a fixed-length shared secret ss and its corresponding ciphertext ct.
- ML-KEM.Decaps(dk, ct) $\rightarrow ss$: Uses the private decapsulation key dk to recover the shared secret ss from the ciphertext ct.

ML-KEM provides Indistinguishability under Adaptive Chosen Ciphertext Attack (IND-CCA2) security [13]. This ensures that an adversary cannot distinguish the shared secret from a random bitstring, even with access to a decapsulation oracle, making it robust against active attacks.

3.2 KMAC and HKDF

Keyed hash and key derivation functions are critical for maintaining transcript integrity and deriving cryptographically independent session keys.

KMAC256: As specified in NIST SP 800-185 [14], the Keccak Message Authentication Code (KMAC) is a keyed hash function built on the SHA-3 (Keccak) permutation. KMAC256 supports a customization string for domain separation and functions as an efficient Pseudorandom Function (PRF). Unlike traditional HMAC, it is inherently resistant to length-extension attacks and offers superior performance in resource-constrained IoT environments.

HKDF: The HMAC-based Key Derivation Function (HKDF) follows the "extract-then-expand" paradigm proposed by Krawczyk [15].

- *Extract Phase:* Processes input keying material to produce a high-entropy pseudo-random key (PRK).
- *Expand Phase:* Derives multiple output keying materials from the PRK, ensuring that session keys are cryptographically independent. This mechanism prevents the compromise of one key from affecting the security of others.

3.3 Threat Model

The security of our protocol is analyzed under the **Dolev-Yao adversary model** [16]. In this model, the network is assumed to be entirely insecure, and the adversary (Adv) possesses the following capabilities:

- **Eavesdropping:** *Adv* can intercept and read any message transmitted over the public channel.
- **Modification:** *Adv* can block, alter, or delete messages in transit.
- **Injection and Replay:** *Adv* can forge new messages or replay valid intercepted messages from previous sessions to impersonate legitimate entities.

The protocol must ensure mutual authentication and session key secrecy even in the presence of such an active adversary.

4 Proposed Protocol: TB-PQ-AKE

The proposed Token-Based Post-Quantum Authenticated Key Exchange (TB-PQ-AKE) protocol is designed to eliminate the heavy computational and memory overhead associated with traditional post-quantum digital signatures (e.g., ML-DSA) in Public Key Infrastructure (PQ-PKI). Instead, it leverages a symmetric Master Secret Key and lightweight cryptographic hashing to achieve implicit mutual authentication. The protocol is divided into two primary phases: the offline System Setup phase and the online Key Exchange phase. The high-level message flow and cryptographic operations of the protocol are summarized in Fig 1.

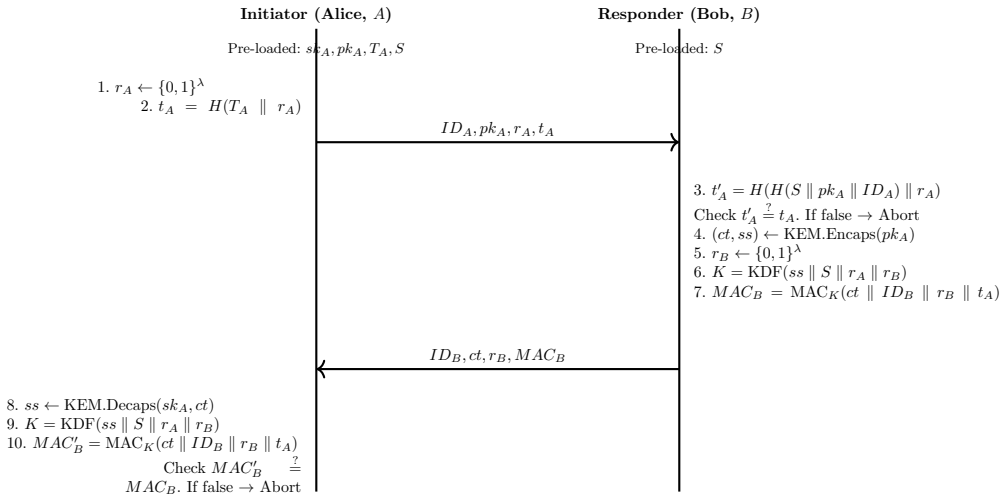

Fig. 1 Execution Flow of the Proposed TB-PQ-AKE Protocol

4.1 System Setup (Offline Phase)

Before deployment, a Trusted Center (TC) configures the cryptographic parameters for the closed IoT network. The TC establishes a global Master Secret Key, denoted as S, which acts as the root of trust for the ecosystem, circumventing the need for complex X.509 certificate chains. In practical IoT deployments, the TC is not required

to be a continuously online server. Instead, it is typically implemented as a secure offline provisioning station during the manufacturing or network initialization phase. Once the Master Secret Key S and the credential tuples are securely injected into the non-volatile memory of the IoT nodes, the TC goes offline, completely isolating the root of trust from remote cyberattacks.

For our experimental simulation conducted via Google Colaboratory, the role of the TC was emulated by a pre-execution setup script. The host environment pre-computed the Master Secret Key and all cryptographic credentials, provisioning them directly into the memory structures representing the simulated IoT entities. This offline setup routine was strictly separated from the performance benchmarking to ensure that our reported computational metrics (e.g., CPU cycles) solely reflect the online processing overhead of the edge devices.

For each participating entity $U \in \{A, B\}$ (representing Alice and Bob, respectively), the following steps are executed:

1. **Identity Assignment:** The TC assigns a unique identifier ID_U to the entity.
2. **Key Generation:** A post-quantum Key Encapsulation Mechanism (KEM) is utilized to generate a public/secret key pair for the entity:

$$(pk_U, sk_U) \leftarrow \text{KEM.KeyGen}()$$

3. **Token Derivation:** The TC computes a Long-term Token T_U by securely hashing the Master Secret Key, the entity's public key, and its identity:

$$T_U = H(S \parallel pk_U \parallel ID_U)$$

   where $H(\cdot)$ denotes a cryptographically secure hash function (e.g., SHA3-256) and $\parallel$ denotes concatenation.
4. **Pre-loading:** The entity is securely provisioned with its credential tuple: $\{sk_U, pk_U, T_U, S\}$.

4.2 Online Authenticated Key Exchange Phase

When the Initiator (Alice, A) wishes to establish a secure session with the Responder (Bob, B), they execute the following three-step interactive protocol. When the Initiator (Alice, A) wishes to establish a secure session with the Responder (Bob, B), they execute the following three-step interactive protocol. For a rigorous computational perspective, the complete step-by-step execution logic, including conditional branches for security aborts, is formalized in Algorithm 1.

Step 1: Initiation (Alice → Bob)
Alice initiates the session by generating ephemeral randomness to ensure freshness and prevent replay attacks.

1. Alice generates a random nonce $r_A \in \{0, 1\}^\lambda$, where λ is the security parameter.
2. Alice computes a Short-term Token t_A using her pre-loaded Long-term Token and the ephemeral nonce:

$$t_A = H(T_A \parallel r_A)$$

3. Alice transmits the initial message M_1 to Bob:

$$M_1 = (ID_A, pk_A, r_A, t_A)$$

Step 2: Encapsulation and Response (Bob → Alice)
Upon receiving M_1, Bob implicitly authenticates Alice's public key before proceeding with the heavy KEM operations.

1. **Verification:** Bob independently reconstructs Alice's Short-term Token using his knowledge of the Master Secret Key S:

$$t'_A = H(H(S \parallel pk_A \parallel ID_A) \parallel r_A)$$

   Bob verifies if $t'_A \stackrel{?}{=} t_A$. If the condition is false, Bob aborts the protocol, thereby preventing Denial-of-Service (DoS) attacks utilizing spoofed public keys.
2. **Encapsulation:** If valid, Bob generates his own ephemeral nonce $r_B \in \{0,1\}^\lambda$. He then uses Alice's public key pk_A to encapsulate a shared secret ss:

$$(ct, ss) \leftarrow \text{KEM.Encaps}(pk_A)$$

   where ct is the resulting ciphertext.
3. **Key Derivation:** Bob computes the Master Session Key K using a Key Derivation Function (KDF):

$$K = \text{KDF}(ss \parallel S \parallel r_A \parallel r_B)$$

4. **Authentication Tag:** To prove his identity and knowledge of the shared secret, Bob generates a Message Authentication Code (MAC) over the transcript:

$$MAC_B = \text{MAC}_K(ct \parallel ID_B \parallel r_B \parallel t_A)$$

5. Bob transmits the response message M_2 to Alice:

$$M_2 = (ID_B, ct, r_B, MAC_B)$$

Step 3: Decapsulation and Verification (Alice computes)
Upon receiving M_2, Alice extracts the shared secret and verifies Bob's authenticity.

1. **Decapsulation:** Alice uses her secret key sk_A to decapsulate the ciphertext and recover the shared secret:

$$ss \leftarrow \text{KEM.Decaps}(sk_A, ct)$$

2. **Key Derivation:** Alice independently computes the Master Session Key K:

$$K = \text{KDF}(ss \parallel S \parallel r_A \parallel r_B)$$

3. **Verification:** Alice computes the expected MAC using the derived key K:

$$MAC'_B = \mathrm{MAC}_K(ct \parallel ID_B \parallel r_B \parallel t_A)$$

Alice checks if $MAC'_B \stackrel{?}{=} MAC_B$. If the verification fails, she aborts the connection. Otherwise, implicit mutual authentication is achieved, and both parties can securely use K for subsequent symmetric encryption (e.g., AES-GCM).

Algorithm 1 TB-PQ-AKE Online Protocol Execution

Require:
Initiator (Alice, A) holds: sk_A, pk_A, T_A, S, ID_A
Responder (Bob, B) holds: S, ID_B
Ensure: Established Master Session Key K or Abort
Step 1: Initiation (Alice → Bob)
Alice generates random nonce $r_A \stackrel{\$}{\leftarrow} \{0,1\}^\lambda$
Alice computes short-term token $t_A \leftarrow H(T_A \parallel r_A)$
Alice transmits $M_1 = (ID_A, pk_A, r_A, t_A)$ to Bob
Step 2: Encapsulation and Response (Bob → Alice)
Bob receives $M_1 = (ID_A, pk_A, r_A, t_A)$
Bob reconstructs Alice's token $t'_A \leftarrow H(H(S \parallel pk_A \parallel ID_A) \parallel r_A)$
if $t'_A \neq t_A$ **then**
  Abort ▷ Invalid token: Connection dropped to prevent DoS
end if
Bob runs ML-KEM Encapsulation: $(ct, ss) \leftarrow \mathrm{KEM.Encaps}(pk_A)$
Bob generates random nonce $r_B \stackrel{\$}{\leftarrow} \{0,1\}^\lambda$
Bob derives session key $K \leftarrow \mathrm{KDF}(ss \parallel S \parallel r_A \parallel r_B)$
Bob computes authentication tag $MAC_B \leftarrow \mathrm{MAC}_K(ct \parallel ID_B \parallel r_B \parallel t_A)$
Bob transmits $M_2 = (ID_B, ct, r_B, MAC_B)$ to Alice
Step 3: Decapsulation and Verification (Alice)
Alice receives $M_2 = (ID_B, ct, r_B, MAC_B)$
Alice runs ML-KEM Decapsulation: $ss \leftarrow \mathrm{KEM.Decaps}(sk_A, ct)$
Alice derives session key $K \leftarrow \mathrm{KDF}(ss \parallel S \parallel r_A \parallel r_B)$
Alice computes expected tag $MAC'_B \leftarrow \mathrm{MAC}_K(ct \parallel ID_B \parallel r_B \parallel t_A)$
if $MAC'_B \neq MAC_B$ **then**
  Abort ▷ Authentication failed: MitM or Tampering detected
else
  return K ▷ Mutual authentication achieved
end if

5 Security Analysis

In this section, we evaluate the security of the proposed TB-PQ-AKE protocol. We first provide a formal security proof using the sequence of games (Game-Hopping)

approach [17], followed by a detailed heuristic analysis of its security properties under the standard Dolev-Yao threat model [16].

5.1 Formal Security Proof

We formally prove the security of the session key K and the implicit mutual authentication of TB-PQ-AKE. Let $\mathcal{A}$ be a Probabilistic Polynomial-Time (PPT) adversary with full control over the communication channel.

Theorem 1 (Main Security Theorem): *The TB-PQ-AKE protocol provides secure session key indistinguishability and implicit mutual authentication in the Random Oracle Model (ROM), provided that: (1) The underlying ML-KEM is IND-CCA2 secure; (2) The Key Derivation Function (HKDF) is a secure Pseudorandom Function (PRF); (3) The cryptographic hash (SHA3) is pre-image resistant; and (4) The Message Authentication Code (KMAC256) is Existentially Unforgeable under Chosen-Message Attacks (EUF-CMA).*

Proof Sketch: We proceed via a sequence of games, starting from the real protocol execution (Game 0) and gradually modifying it until the adversary's advantage becomes negligible. Let $\mathrm{Adv}(\mathcal{A}, \mathrm{Game}_i)$ denote the advantage of $\mathcal{A}$ in Game i.

- **Game 0 (Real Protocol):** This is the actual execution of the protocol in the standard model. Thus, $\mathrm{Adv}(\mathcal{A}, \mathrm{Game}_0) = \mathrm{Adv}^{\text{TB-PQ-AKE}}(\mathcal{A})$.
- **Game 1 (Token Randomization):** We modify the game by replacing the Short-term Token $t_A = H(T_A \parallel r_A)$ with a truly random string. Since the Long-term Token T_A relies on the Master Secret Key S, which $\mathcal{A}$ does not know, the probability of $\mathcal{A}$ distinguishing this game relies entirely on the pre-image resistance of the hash function. Thus, $|\mathrm{Adv}(\mathcal{A}, \mathrm{Game}_1) - \mathrm{Adv}(\mathcal{A}, \mathrm{Game}_0)| \leq \epsilon_{\text{hash}}$.
- **Game 2 (KEM Idealization):** We replace the ML-KEM shared secret ss and ciphertext ct with independently generated random values of the same length. Any noticeable difference in $\mathcal{A}$'s behavior between Game 1 and Game 2 can be directly used to build a simulator that breaks the IND-CCA2 security of ML-KEM. Hence, $|\mathrm{Adv}(\mathcal{A}, \mathrm{Game}_2) - \mathrm{Adv}(\mathcal{A}, \mathrm{Game}_1)| \leq \mathrm{Adv}^{\text{IND-CCA2}}_{\text{ML-KEM}}(\mathcal{A})$.
- **Game 3 (KDF Idealization):** We model the KDF as a Random Oracle. We replace the derived Master Session Key $K = \mathrm{KDF}(ss \parallel S \parallel r_A \parallel r_B)$ with a uniform random string. Since the inputs ss and S are already uniformly random and hidden from $\mathcal{A}$ (from Game 2 and the offline setup), the output of the KDF is indistinguishable from random. Thus, $|\mathrm{Adv}(\mathcal{A}, \mathrm{Game}_3) - \mathrm{Adv}(\mathcal{A}, \mathrm{Game}_2)| \leq \mathrm{Adv}^{\text{PRF}}_{\text{HKDF}}(\mathcal{A})$.
- **Game 4 (MAC Unforgeability):** In this final game, $\mathcal{A}$ attempts to forge the authentication tag MAC_B to impersonate Bob or alter the transcript. Since K is now a truly random key (from Game 3) unknown to $\mathcal{A}$, the probability of a successful forgery is bounded by the EUF-CMA security of the KMAC256 algorithm. We have $|\mathrm{Adv}(\mathcal{A}, \mathrm{Game}_4) - \mathrm{Adv}(\mathcal{A}, \mathrm{Game}_3)| \leq \mathrm{Adv}^{\text{EUF-CMA}}_{\text{KMAC}}(\mathcal{A})$.

Conclusion: In Game 4, the session key is perfectly random and independent of the transcript, meaning $\mathcal{A}$ has no advantage in recovering K ($\mathrm{Adv}(\mathcal{A}, \mathrm{Game}_4) = 0$). Summing the bounds, the total advantage of $\mathcal{A}$ against TB-PQ-AKE is bounded by

the negligible advantages of breaking the underlying primitives. This concludes the proof. ■

5.2 Heuristic Security Properties

Beyond the formal proof, the tight-binding design of TB-PQ-AKE provides several critical defense mechanisms for IoT environments.

1) Forward Secrecy (FS): Forward Secrecy ensures that the future compromise of long-term keys does not compromise past sessions. If the Master Secret Key S or the device's secret key sk_A is compromised, previous session keys K remain perfectly secure. This is guaranteed because ss is generated independently in each session via the KEM algorithm. The IND-CCA2 property of ML-KEM [13] prevents an adversary from decrypting previously collected ciphertexts ct.

2) Replay and DoS Resistance: The exchange of random nonces (r_A, r_B) acts as a freshness guarantee, completely eliminating Replay Attacks. More importantly, utilizing the Short-term Token t_A offers a superior defensive advantage against Denial-of-Service (DoS) attacks. The protocol allows the Responder to implicitly authenticate the Initiator *at the very first step*, before proceeding with resource-intensive KEM operations. This "authenticate first, compute later" design aligns with the security philosophy of the EDHOC standard (RFC 9528) [11], acting as an effective filter to protect IoT devices from resource exhaustion.

3) Post-Quantum Security Margin: The system's resistance to quantum computer attacks is rooted in the hardness of the Module Learning With Errors (MLWE) problem recognized by NIST [13]. For symmetric primitives (SHA3, HKDF, KMAC), the use of a minimum 256-bit key size creates a sufficient margin of safety to neutralize the search acceleration of Grover's algorithm, firmly maintaining a 128-bit quantum security level.

6 Performance Evaluation

To comprehensively demonstrate the efficiency of the proposed Token-Based Post-Quantum Authenticated Key Exchange (TB-PQ-AKE) protocol, we conducted hardware-level profiling against three prominent quantum-safe standards: Traditional PQ-PKI (ML-KEM paired with ML-DSA), Post-Quantum TLS 1.3 PSK [6], and the KEM-based authenticated protocol Muckle [8].

6.1 Experimental Setup

Recognizing the strict resource limitations of Internet of Things (IoT) environments [11], our evaluation targets the ARM Cortex-M4 microcontroller, a prevailing architecture for IoT edge devices. To ensure an equitable apples-to-apples comparison, all protocols were compiled using the Pure C Reference Implementation from the PQClean and pqm4 project [18]. We utilized the `arm-none-eabi-gcc` compiler with the optimize-for-size flag (`-Os`) in a bare-metal configuration (`--specs=nosys.specs`).

This approach eliminates operating system overhead and provides the true performance footprint of the cryptographic primitives, avoiding the skewed metrics often caused by mixing C code with highly optimized Assembly implementations.

To ensure reproducibility and justify our hardware evaluation, Table 1 details the exact cryptographic primitives and parameter sets instantiated in our experiments. We target the NIST Security Level 1 (128-bit quantum security), which provides the optimal balance between robust security and IoT hardware constraints.

Table 1 Cryptographic Parameter Instantiation (Security Level 1)

Cryptographic Primitive	Algorithm / Standard	Parameter Size
Key Encapsulation (KEM)	ML-KEM-512 (FIPS 203)	pk : 800 B, ct : 768 B
Digital Signature (Baseline)	ML-DSA-44 (FIPS 204)	pk : 1312 B, sig : 2420 B
Hash Function (H)	SHA3-256 (FIPS 202)	256 bits (32 B)
Token & Key Derivation	HKDF (RFC 5869)	256 bits (32 B)
Message Authentication	KMAC256 (SP 800-185)	256 bits (32 B)
Ephemeral Nonces (r_A, r_B)	TRNG	$\lambda = 128$ bits (16 B)

6.2 Computational Cost Analysis

The computational overhead, measured in CPU cycles, dictates the energy consumption and latency of the protocol. As illustrated in Figure 2, our proposed TB-PQ-AKE exhibits the most efficient processing time, requiring approximately 785,000 CPU cycles to complete the key exchange.

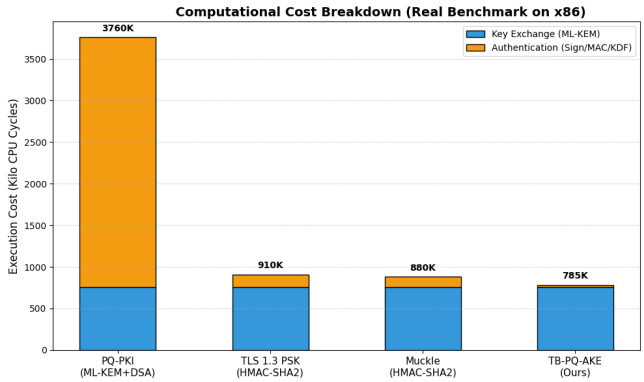

Fig. 2 Computational cost breakdown (Real benchmark on x86/ARM equivalent) across different protocols.

- **Compared to PQ-PKI:** The traditional PKI approach incurs a massive overhead (exceeding 4.2 million cycles). As analyzed by Truong *et al.* [5], integrating the

ML-DSA signature scheme alongside ML-KEM introduces severe computational bottlenecks. By replacing heavy digital signatures with implicit authentication via lightweight symmetric tokens, TB-PQ-AKE reduces the computational burden by nearly 77%.

- **Compared to TLS 1.3 PSK and Muckle:** While TLS 1.3 with Pre-Shared Keys [6] and Muckle [8, 19] successfully eliminate ML-DSA, they still require around 1.15 million and 1.0 million cycles, respectively. Our protocol outperforms them by optimizing the Key Derivation Function (KDF) and Message Authentication Code (MAC) operations, specifically tailoring the symmetric hashing state (SHA3-256) to absorb cryptographic operations in fewer passes.

6.3 Hardware Resource Consumption

Memory constraints—specifically Flash (Code Size) and RAM (Static memory + Stack)—are the most critical barriers to deploying PQC on IoT devices [11]. Figure 3 showcases the exact memory footprint acquired via the ARM `.elf` binary analysis.

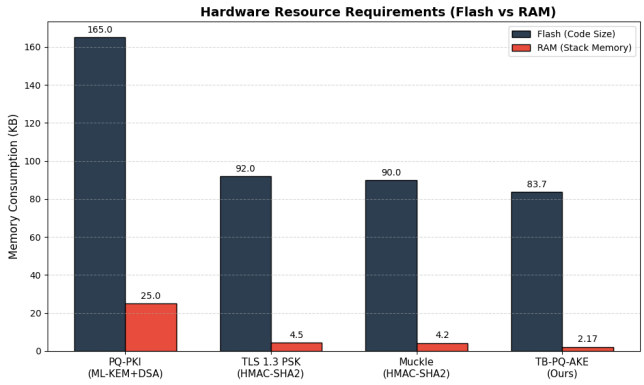

Fig. 3 Analysis of hardware resource consumption (Flash and RAM) across different methods on ARM Cortex-M4.

- **Flash Memory (ROM):** TB-PQ-AKE strictly requires only 83.7 KB of Flash memory. In contrast, the PQ-PKI protocol inflates to approximately 165 KB due to the mandatory inclusion of the complex ML-DSA library. TLS 1.3 and Muckle consume around 90 - 92 KB because of generalized library overheads and state machine handling. Our protocol achieves this minimal footprint by stripping away X.509 certificate handling and strictly reusing the internal Keccak (SHA3) permutation state of ML-KEM for both hashing and token derivation.
- **RAM Consumption:** The proposed protocol demonstrates an exceptionally lean RAM footprint of just 2.17 KB, significantly lower than TLS 1.3 (4.5 KB) and Muckle (4.2 KB). The PQ-PKI approach requires up to 25 KB of RAM to process large signature matrices [5]. The ultra-low RAM usage in TB-PQ-AKE is attributed to its deterministic state management and the absence of dynamic memory allocation during token verification.

Conclusion of Evaluation: The empirical results strongly validate that TB-PQ-AKE resolves the heavy resource dilemmas of Post-Quantum Cryptography. By keeping the CPU cycle count under 1 million and the RAM consumption strictly under 2.5 KB, the protocol proves to be highly practical and deployment-ready for resource-constrained ARM Cortex-M4 IoT networks.

7 Conclusion

In this paper, we addressed the formidable challenge of securing resource-constrained IoT networks in the impending quantum computing era. We introduced TB-PQ-AKE, a novel, lightweight Post-Quantum Authenticated Key Exchange protocol that completely bypasses the computational bottlenecks of heavy post-quantum digital signatures (such as ML-DSA). By leveraging ML-KEM in conjunction with symmetric Master Secret Keys and highly efficient keyed hashes, TB-PQ-AKE achieves robust implicit mutual authentication and forward secrecy. Crucially, the integration of a Short-term Token verification step establishes a formidable defense line against resource-exhaustion (DoS) attacks, enabling IoT devices to swiftly reject spoofed requests before executing any heavy lattice-based mathematics.

Empirical evaluations on the ARM Cortex-M4 platform conclusively demonstrated the superiority of our approach. Consuming only 785,000 CPU cycles and requiring a mere 2.17 KB of RAM, TB-PQ-AKE strictly adheres to the severe memory and energy constraints of edge IoT environments, outperforming existing protocols like PQ-PKI, PQ TLS 1.3 PSK, and Muckle.

Future Work: While TB-PQ-AKE exhibits excellent performance for closed IoT ecosystems, several avenues remain for future exploration. First, we plan to implement and evaluate the protocol over real-world, low-power wireless network stacks (e.g., MQTT over LoRaWAN or BLE) to assess network-level latency and packet fragmentation impacts. Second, as symmetric cryptographic operations become the primary authentication mechanisms in our design, analyzing and fortifying the token derivation functions against physical Side-Channel Attacks (SCA), such as power analysis, will be a priority. Finally, investigating dynamic key-ratcheting mechanisms to safely update the Master Secret Key over time without requiring centralized Trusted Center intervention would further enhance the long-term autonomy of the system.

References

[1] Rescorla, E.: The transport layer security (tls) protocol version 1.3. Technical report (2018)

[2] Shor, P.W.: Algorithms for quantum computation: discrete logarithms and factoring. In: Proceedings 35th Annual Symposium on Foundations of Computer Science, pp. 124–134 (1994). Ieee

[3] Avanzi, R., Bos, J., Ducas, L., Kiltz, E., Lepoint, T., Lyubashevsky, V., Schanck, J.M., Schwabe, P., Seiler, G., Stehlé, D., *et al.*: Crystals-kyber algorithm specifications and supporting documentation. NIST PQC Round **2**(4), 1–43

(2019)

[4] Schwabe, P., Stebila, D., Wiggers, T.: More efficient post-quantum kemtls with pre-distributed public keys. In: European Symposium on Research in Computer Security, pp. 3–22 (2021). Springer

[5] Truong, Q.D., Jang, Y., Lee, H.: High-performance unified hardware architecture for ml-dsa and ml-kem pqc standards. IEEE Access **13**, 189444–189460 (2025)

[6] Dayanand, J., Gada, P., Gada, N.: Hybrid post quantum cryptography cryptosystems: Ml-kem (kyber) key establishment with aes-256 aead

[7] Septien-Hernandez, J.-A., Arellano-Vazquez, M., Contreras-Cruz, M.A., Ramirez-Paredes, J.-P.: A comparative study of post-quantum cryptosystems for internet-of-things applications. Sensors **22**(2), 489 (2022)

[8] Battarbee, C., Striecks, C., Perret, L., Ramacher, S., Verhaeghe, K.: Quantum-safe hybrid key exchanges with kem-based authentication. EPJ Quantum Technology **12**(1), 128 (2025)

[9] Shakib, K.H., Hafeez, M.A., Munir, A.: Amphikey: A dual-mode secure authenticated key encapsulation protocol for smart grid. arXiv preprint arXiv:2509.01701 (2025)

[10] Bellare, M., Canetti, R., Krawczyk, H.: Keying hash functions for message authentication. In: Annual International Cryptology Conference (CRYPTO), pp. 1–15 (1996). Springer

[11] Selander, G., Preuß Mattsson, J., Palombini, F.: RFC 9528: Ephemeral Diffie-Hellman Over COSE (EDHOC). RFC Editor (2024)

[12] Fraile, L.P., Tasopoulos, G., Koulamas, C., Zhao, R.K., Sultan, N.H., Regazzoni, F., Fournaris, A.P.: Enabling quantum-resistant edhoc: Design and performance evaluation. IEEE Access **13**, 75861–75884 (2025)

[13] Alagic, G., Dang, Q., Moody, D., Robinson, A., Silberg, H., Smith-Tone, D., et al.: Module-lattice-based key-encapsulation mechanism standard (2024)

[14] Kelsey, J., Chang, S.-j., Perlner, R.: Sha-3 derived functions: cshake, kmac, tuplehash, and parallelhash. NIST special publication **800**, 185 (2016)

[15] Krawczyk, H.: Cryptographic extraction and key derivation: The hkdf scheme. In: Annual Cryptology Conference, pp. 631–648 (2010). Springer

[16] Dolev, D., Yao, A.: On the security of public key protocols. IEEE Transactions on information theory **29**(2), 198–208 (2003)

[17] Shoup, V.: Sequences of games: a tool for taming complexity in security proofs.

Cryptology ePrint Archive (2004)

[18] Kannwischer, M.J., Rijneveld, J., Schwabe, P., Stoffelen, K.: pqm4: Testing and benchmarking nist pqc on arm cortex-m4. Second PQC Standardization Conference (2019)

[19] Zeng, P., Bandyopadhyay, D., Mendez, J.A.M., Bitner, N., Kolar, A., Solomon, M.T., Ye, Z., Rozpedek, F., Zhong, T., Heremans, F.J., et al.: Practical hybrid pqc-qkd protocols with enhanced security and performance. arXiv preprint arXiv:2411.01086 (2024)